

Lab 10

GANs and Generative AI Assignments

Assignment 1

Design and implement a Generative Adversarial Network (GAN) to generate realistic human face images. Use the **CelebADataset** from Kaggle.

- Preprocess and prepare the dataset.
- Build and train the GAN model with clearly defined Generator and Discriminator architectures.
- Analyze the training process and evaluate the quality of the generated images.
- Discuss challenges and possible improvements.

Dataset link: <https://www.kaggle.com/datasets/jessicali9530/celeba-dataset>

About Dataset

Overall

- 202,599 number of face images of various celebrities
- 10,177 unique identities, but names of identities are not given
- 40 binary attribute annotations per image
- 5 landmark locations

Data Files

- **img_align_celeba.zip**: All the face images, cropped and aligned
- **list_eval_partition.csv**: Recommended partitioning of images into training, validation, testing sets
- **list_bbox_celeba.csv**: Bounding box information for each image. "x_1" and "y_1" represent the upper left point coordinate of bounding box.
- **list_landmarks_align_celeba.csv**: Image landmarks and their respective coordinates. There are 5 landmarks: left eye, right eye, nose, left mouth, right mouth
- **list_attr_celeba.csv**: Attribute labels for each image. There are 40 attributes. "1" represents positive while "-1" represents negative

Data Preparation

- **Download Dataset:**

Use the full Kaggle link: <https://www.kaggle.com/datasets/jessicali9530/celeba-dataset>

- **Preprocess:**

- Resize all images to a fixed size (e.g., 64×64 or 128×128 pixels).
- Normalize pixel values to $[-1, 1]$ (common practice for GANs).

Build the GAN Architecture

- Generator:**

- ☐ Input: A noise vector.
- ☐ Architecture: Series of dense layers followed by transposed
- ☐ Convolution layers (Conv2DTranspose) with batch normalization
- ☐ Activation (ReLU/tanh).

- Discriminator:**

- ☐ Input: An image (real or generated).
- ☐ Architecture: Convolutional layers with leaky ReLU activations,
- ☐ Single output node (sigmoid activation) to classify real vs. fake.

Model Training

- Initialize Generator and Discriminator models
 - ❑ `generator = build_generator(input_dim=100)`
 - ❑ `discriminator = build_discriminator(input_shape=(64, 64, 3))`
- Compile models with optimizers and loss functions
 - ❑ `discriminator.compile(optimizer='adam', loss='binary_crossentropy')`
 - ❑ `combined_model = combine_models(generator, discriminator)`
 - ❑ `combined_model.compile(optimizer='adam', loss='binary_crossentropy')`

Assignment 2

Using the ***Chest X-ray Pneumonia Dataset***, develop a Generative Adversarial Network (GAN) to enhance the quality of chest X-ray images by converting low-quality scans into high-quality versions. Your task involves:

- Downloading and preprocessing the dataset by resizing images,
- Normalizing pixel values, and artificially degrading some images (e.g., using Gaussian blur or downscaling) to create a low-quality subset.
- Implement a GAN architecture consisting of a generator, which enhances the degraded images, and a discriminator, which distinguishes between real high-quality and generated images.
- Train the GAN using an adversarial loss function and evaluate its performance using image quality metrics such as Structural Similarity Index (SSIM) and Peak Signal-to-Noise Ratio (PSNR).
- Finally, apply the trained GAN model to enhance a set of test X-ray images, compare the results with the original high-quality images, and analyze the model's effectiveness, limitations, and potential improvements.

Dataset link: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

Dataset Description

Classes:

1. **Normal:** 1,341 images of healthy lungs.
2. **Pneumonia:** 3,875 images of lungs infected with pneumonia.
 1. Subtypes: Bacterial (2,538) and Viral (1,347).

Image Specifications:

1. **Format:** JPEG.
2. **Resolution:** Variable (e.g., 1000x700px to 2000x2000px).
3. **Color:** Grayscale (stored as RGB with identical channels).

Classification: Binary (normal vs. pneumonia) or multiclass (bacterial vs. viral).



Step 1: Dataset Preparation

- **Preprocess Images**

- Convert all images to grayscale (if needed).
- Resize images to a fixed shape (e.g., 256x256).
- Normalize pixel values to [0,1] for stable training.

- **Simulating Low-Quality X-rays**

- Apply Gaussian blur or downsampling to generate degraded images.
- Store paired data: (*low-quality image*, *high-quality image*).

Step 2: Building the GAN Model

Generator

- Uses a convolutional network with U-Net or ResNet blocks.
- Converts a low-quality image into a high-quality image.

Discriminator

- Uses a CNN to classify whether an image is real (from the dataset) or generated (from the generator).
- Helps the generator improve by providing feedback.

Step 3: Model Training

Initialize GAN Components

- Load dataset with paired low-quality and high-quality images.
- Define generator and discriminator networks.

Define Loss Functions

- **Adversarial Loss** (Binary Cross-Entropy): Ensures the generator creates realistic images.
- **L1 Loss / Mean Squared Error (MSE)**: Encourages structural similarity between generated and real images.

Algorithm for build_generator()

- Input:** Low-quality X-ray image of shape (256, 256, 1).

- Encoding (Downsampling):**

- Apply convolutional layers with increasing filters ($64 \rightarrow 128 \rightarrow 256$).
- Use **stride-2 convolutions** for downsampling.

- Bottleneck:**

- Use a **512-filter convolution layer** to capture deep features.

- Decoding (Upsampling):**

- Apply **upsampling + transposed convolutions** ($256 \rightarrow 128 \rightarrow 64$).
- Use **skip connections** (optional for better reconstruction).

- Output Layer:**

- Apply a final 1-channel convolution with **tanh activation** to generate the enhanced image.

Algorithm for build_discriminator()

- **Input:** X-ray image of shape (256, 256, 1).
- **Feature Extraction:**
 - Apply **3 convolutional layers** with increasing filters (64 → 128 → 256).
 - Use **stride-2 convolutions** for downsampling.
- **Flatten and Fully Connected Layer:**
 - Flatten extracted features.
 - Apply a dense layer with **512 neurons** and ReLU activation.
- **Output Layer:**
 - Apply a final **dense layer** with **sigmoid activation** (0 = fake, 1 = real).

Evaluate and Test

- Compare generated images with real images.
- Save the best-performing model.
- Testing
 - Load the trained generator.
 - Input new low-quality images and generate enhanced versions.
 - Compare results using visual inspection and quality metrics.

Thank you

